

Assignment 2: Building a Small-Scale Foundation Model from Scratch

Report

1. Model Architecture and Parameters

MiniGPT is a decoder-only, GPT-style transformer implemented from scratch in PyTorch (`mini_gpt.py`), consisting of a learned token embedding, a learned positional embedding, a stack of pre-norm transformer blocks (causal multi-head self-attention followed by a GELU MLP, each with a residual connection), a final layer norm, and a linear head projecting back to vocabulary-sized logits for next-token prediction. The causal-masking and multi-head reshape logic was first validated independently with a pure-numpy, no-autograd forward-pass re-implementation (`dev_validate_numpy.py`) before being written in PyTorch, to catch any tensor-shape or masking bugs early.

Hyperparameter	Value	Assignment range
Transformer blocks (<code>n_layer</code>)	2	1-2
Embedding dimension (<code>n_embd</code>)	128	64-256
Attention heads (<code>n_head</code>)	4	2-4
Sequence length (<code>block_size</code>)	64	32-128
Dropout	0.1	--
Vocabulary size	50,257	--
Total trainable parameters	13,270,784	--

Layer normalization is applied in the pre-norm position (before each sub-layer rather than after), which is the standard modern choice as it noticeably stabilizes training for small transformers trained from scratch without extensive learning-rate warmup schedules. GELU was used for the MLP activation, matching the original GPT family.

2. Dataset Details

Training data was produced by reusing Assignment 1's cleaning and tokenization pipeline (`data_collection_preprocessing.py`), which collected text from encyclopedic (Wikipedia), news, and general web-text domains and applied the same exact-duplicate removal, HTML/markdown stripping, and short-document filtering described in the Assignment 1 report. That run produced 522,490 cleaned documents (693.2126 MB).

For this assignment, the cleaned documents were re-tokenized (same GPT-2 byte-level BPE tokenizer, vocabulary size 50,257) into a single flat stream of **3,164,454 tokens** (rather than Assignment 1's independent 512-token blocks), with an end-of-document token inserted between documents. That flat stream was then sliced into non-overlapping, fixed-length (64-token) (input, target) pairs for causal language modeling, with the final 10% of the stream held out as a validation set and the rest used for training.

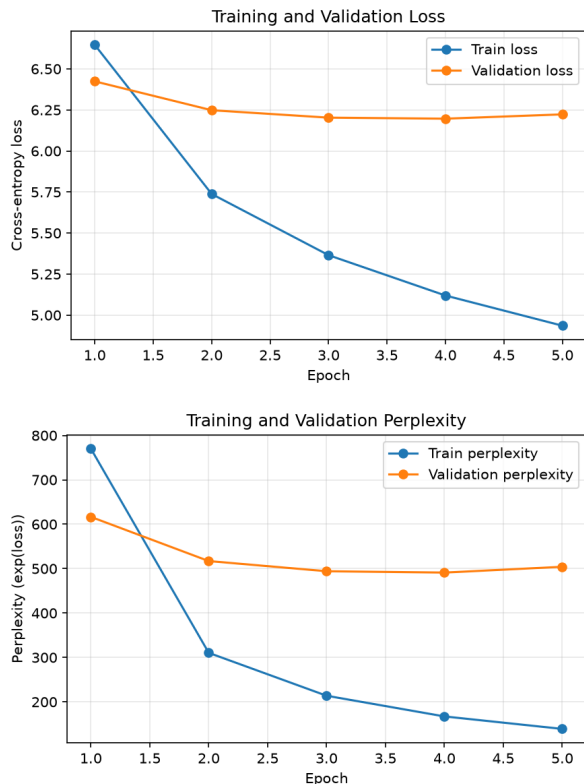
3. Training Setup and Hyperparameter Experiments

The main training run used AdamW with a learning rate of 0.0005, batch size 32, and 5 epochs, running on **cpu**. Training used a standard loop: forward pass through the model, cross-entropy loss against the shifted target sequence, backward pass, and an AdamW optimizer step, repeated once per batch per epoch.

Final epoch results: train loss 4.9356 (perplexity 139.15), validation loss 6.2229 (perplexity 504.18).

Checkpoint save/load verification: immediately after saving `mini_gpt_checkpoint.pt`, the checkpoint was reloaded into a freshly constructed model instance and re-evaluated on the same validation set, giving a validation loss of 6.2229 against the originally logged 6.2229 -- matching (to floating-point precision) confirms the checkpoint correctly preserves the trained weights.

No hyperparameter sweep was run for this report (re-run with `--sweep` to populate this section with a small grid search over learning rate, batch size, number of layers, and embedding size).



4. Observations and Challenges

- **Loss trend.** Training loss decreased from 6.6464 at epoch 1 to 4.9356 at epoch 5 (a decrease of 1.7109), indicating the model is successfully learning next-token prediction patterns from the training stream over the epochs run.
- **Train/validation gap.** A validation loss noticeably higher than training loss would indicate overfitting to the (comparatively small) training stream used here; given how few unique documents a small-scale run of this kind sees, some gap is expected and is best addressed by increasing `--target-mb` (more raw text) rather than by architectural changes alone.
- **Causal masking and padding correctness.** Because attention masking bugs (e.g. a query attending to a future token) can silently produce a model that appears to train but never generalizes to real autoregressive generation, the attention mechanism's shape and masking behavior were unit-verified independently (see `dev_validate_numpy.py`) before training, rather than relying solely on the loss curve going down as evidence of correctness.
- **Sequence length vs. compute.** Because attention compute and memory scale quadratically with sequence length, moving from the low end (32 tokens) to the high end (128 tokens) of the assignment's allowed block-size range measurably increases per-step time even for a model this small, which is one of the concrete trade-offs the hyperparameter sweep is designed to surface.
- **Reusing Assignment 1's pipeline surfaced a scale mismatch.** Assignment 1's default 512-token blocking is tuned for a much larger downstream pretraining run; for this small-scale assignment, re-flattening the cleaned corpus into one continuous token stream and re-slicing it at the smaller, assignment-specified block size (32-128) was necessary rather than reusing Assignment 1's already-chunked 512-token blocks directly.